



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 9. Отладка программ и тестирование

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Режим отладки
-  Типы отладчиков и профайлеров
-  Методологии тестирования ПО
-  Классификация тестов
-  Юнит-тестирование
-  Средства автоматизации тестирования

Режим отладки

- 🌀 Режим отладки (debug) – специальный режим выполнения программы, позволяющий выявлять причины аномального поведения программы
 - 🌀 Может быть использован для изучения работы чужого кода
- 🌀 Отладка выполняется в среде специальных программ отладчиков (debugger)
 - 🌀 Могут запускаться самостоятельно из командной строки
 - pdb – встроенный отладчик для Python
 - 🌀 Обычно доступны в составе интегрированных сред разработки
- 🌀 В режиме отладки доступны:
 - 🌀 Пошаговое выполнение программы
 - 🌀 Добавление точек останова (breakpoint)
 - 🌀 Просмотр текущих значений переменных
 - 🌀 Изменение текущих значений переменных

Пример отладки кода

 Функция вычисляет среднее арифметическое элементов списка

```
def average(numbers):  
    total = 0  
    for num in numbers:  
        total += num  
    average = total / len(numbers)  
    return average
```

 При запуске следующего кода возникает ошибка

```
l = [0,1,2,3,4,5,6,7,8,9]  
  
while l:  
    del l[-1]  
    print(average(l))  
  
# ZeroDivisionError: division by zero
```

Добавление точек останова в код



Встроенная функция breakpoint()

- Вызов функции вставляется в код в место, где нужно переключиться в режим отладки
- Запускает встроенный отладчик Python **pdb**
- Выполнение программы останавливается на строке с вызовом функции

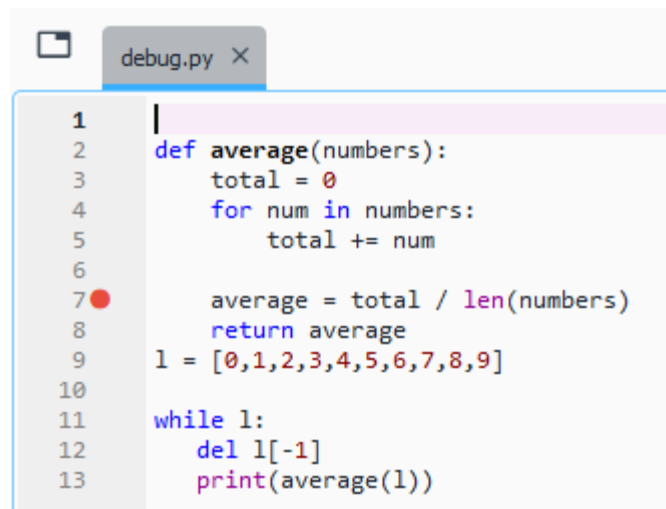
The screenshot shows a Python IDE with a file named `debug.py`. The code defines a function `average` and a list `l`. A breakpoint is set on line 6, which is highlighted with a pink background and the comment `# Точка останова`. The variable inspector on the right shows the state of the program at the breakpoint.

```
1
2 def average(numbers):
3     total = 0
4     for num in numbers:
5         total += num
6 breakpoint() # Точка останова
7     average = total / len(numbers)
8     return average
9 l = [0,1,2,3,4,5,6,7,8,9]
10
11 while 1:
12     del l[-1]
13     print(average(l))
```

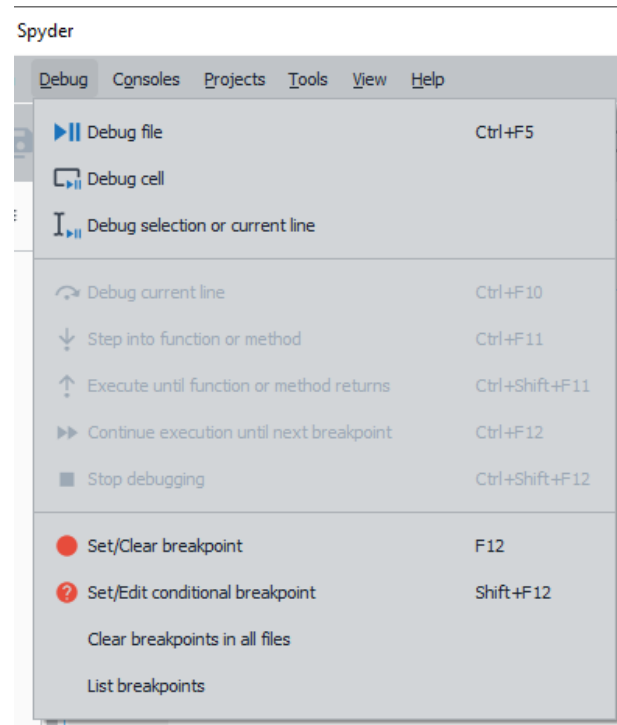
Name	Type	Size	Value
l	list	9	[0, 1, 2, 3, 4, 5, 6, 7, 8]
num	int	1	8
numbers	list	9	[0, 1, 2, 3, 4, 5, 6, 7, 8]
total	int	1	36

Визуальное добавление точек останова

- Поддерживается всеми основными IDE
- Точка может быть добавлена на любую строку программы, содержащую исполняемый код
- Чтобы точка сработала, программа должна быть запущена в режиме отладки



```
1 |
2 def average(numbers):
3     total = 0
4     for num in numbers:
5         total += num
6
7 ●     average = total / len(numbers)
8     return average
9 l = [0,1,2,3,4,5,6,7,8,9]
10
11 while 1:
12     del l[-1]
13     print(average(l))
```

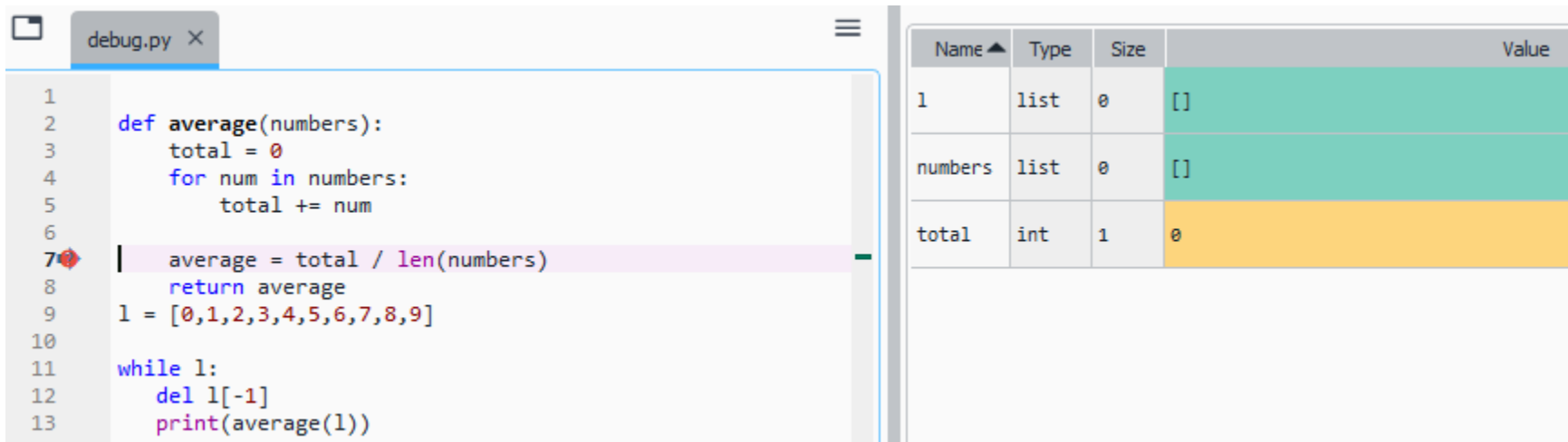
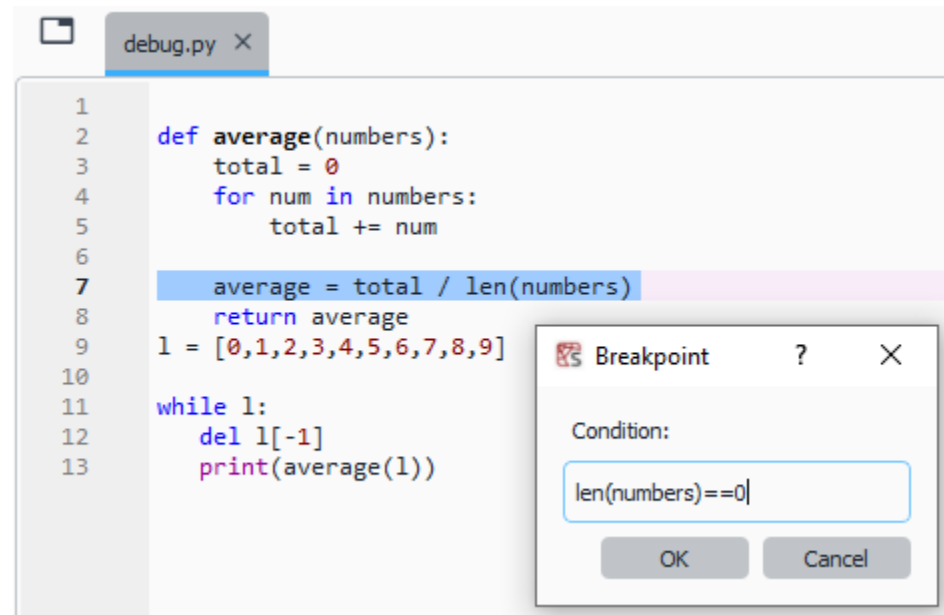


Точки останова с условием



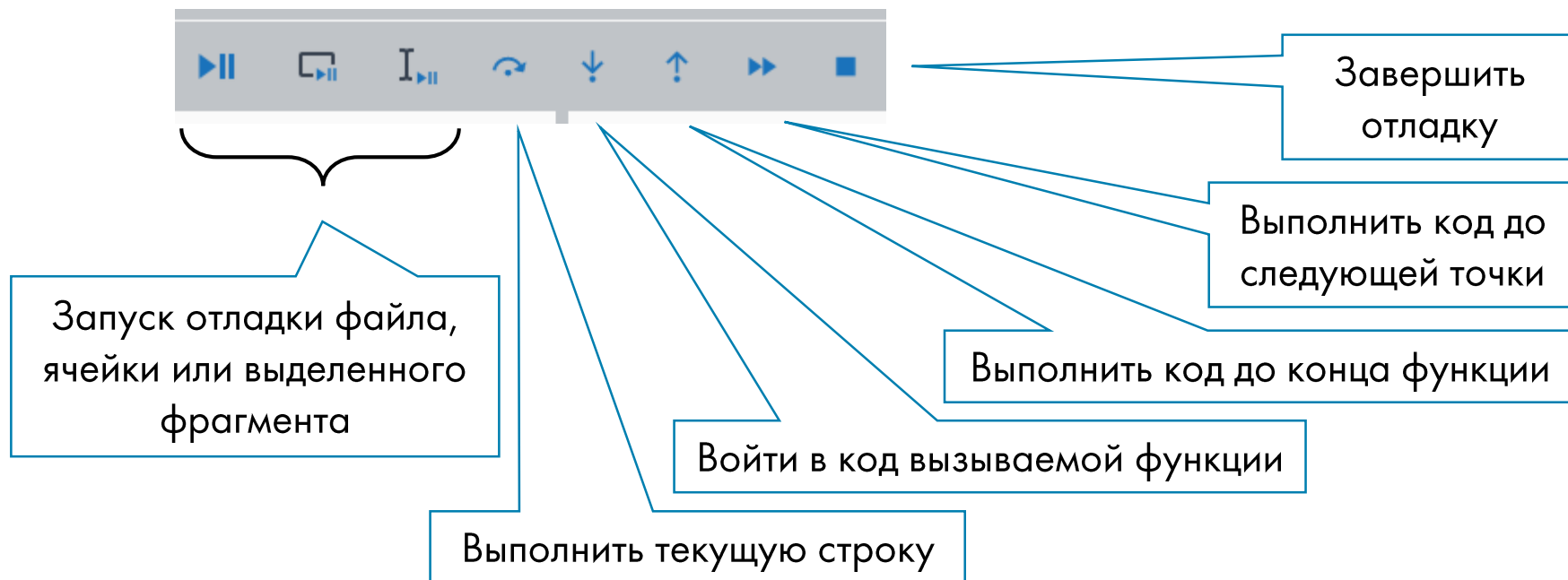
Выполнение программы останавливается на точке только если условие выполняется

- Можно пропустить часть итераций цикла или следить за определенной переменной



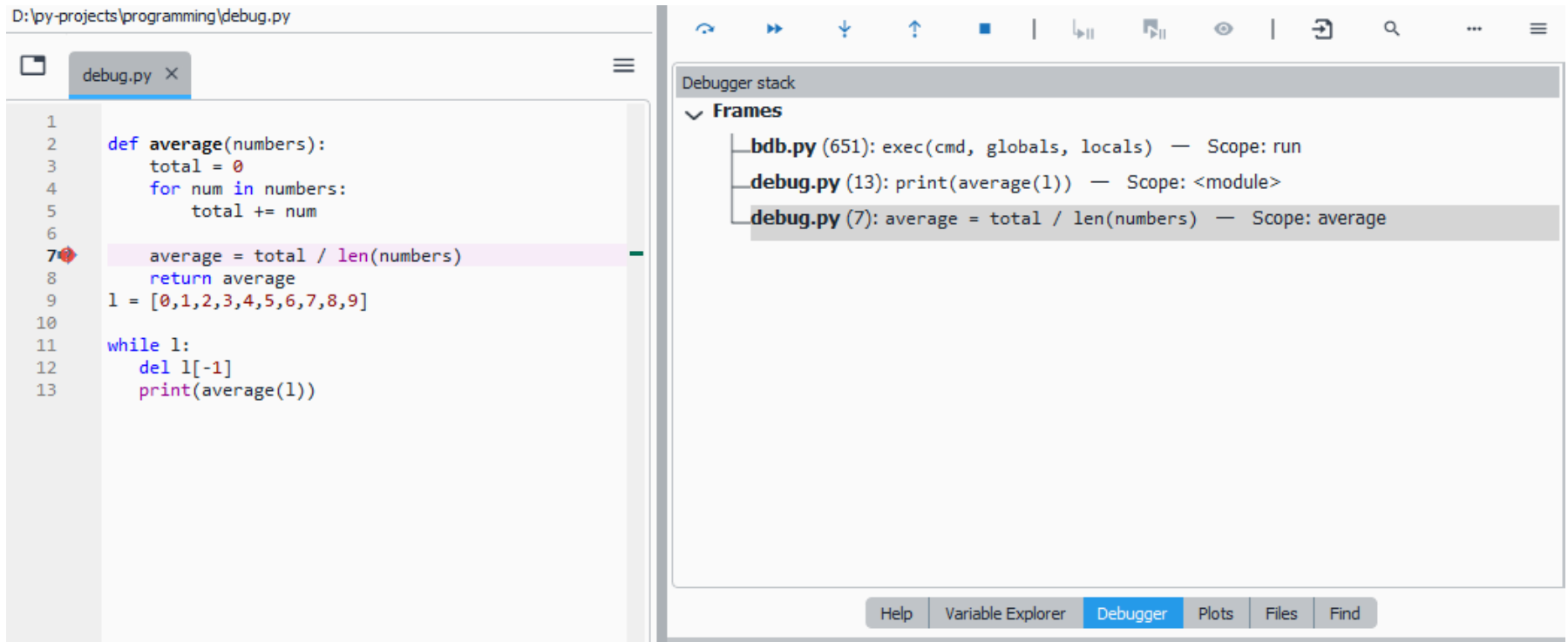
Пошаговое выполнение

- После остановки выполнения программы на точке можно продолжить выполнение программы по шагам, перейти к следующей точке или завершить отладку



Представление Debugger

 Показывает состояние программы, включая стек вызовов



Список точек останова

D:\py-projects\programming\debug.py

debug.py x

```
1
2 def average(numbers):
3     total = 0
4     for num in numbers:
5         total += num
6
7 | average = total / len(numbers)
8     return average
9 l = [0,1,2,3,4,5,6,7,8,9]
10
11 while l:
12     del l[-1]
13     print(average(l))
```

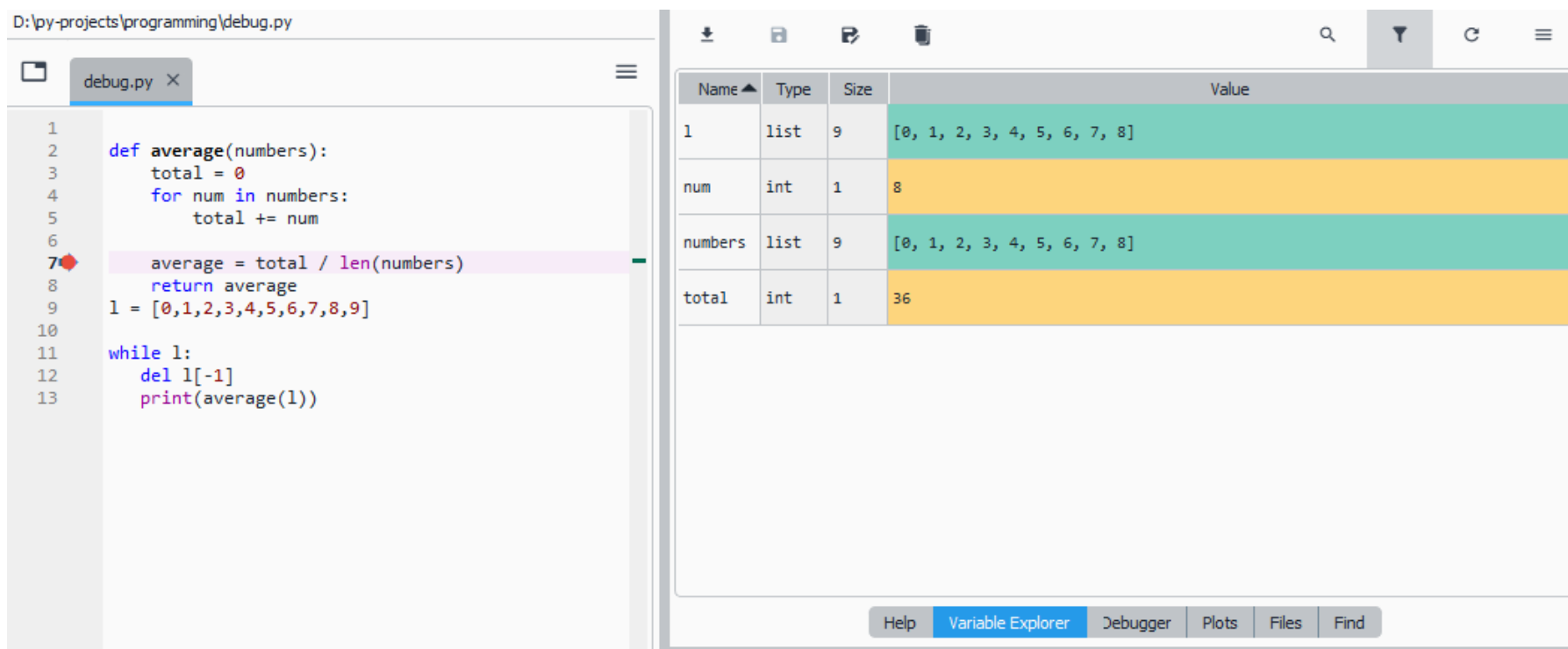
File Line Condition

debug.py	7	len(numbers)==0
main.py	4	

Help Variable Explorer Debugger Plots Files Find

Представление Variable Explorer

- Позволяет просматривать значения переменных на текущем шаге выполнения программы



The screenshot displays a Python IDE with a code editor on the left and a Variable Explorer window on the right. The code editor shows a file named `debug.py` with the following content:

```
1
2 def average(numbers):
3     total = 0
4     for num in numbers:
5         total += num
6
7     average = total / len(numbers)
8     return average
9 l = [0,1,2,3,4,5,6,7,8,9]
10
11 while 1:
12     del l[-1]
13     print(average(l))
```

The Variable Explorer window shows the current state of variables:

Name	Type	Size	Value
l	list	9	[0, 1, 2, 3, 4, 5, 6, 7, 8]
num	int	1	8
numbers	list	9	[0, 1, 2, 3, 4, 5, 6, 7, 8]
total	int	1	36

The Variable Explorer window includes a search bar, a filter icon, and a refresh icon. The bottom of the window shows a tabbed interface with the following tabs: Help, Variable Explorer, Debugger, Plots, Files, and Find.

Профилирование

- 📡 Профилирование (profiling) – сбор характеристик работы программы для ее последующей оптимизации:
 - ⦿ Время выполнения отдельных функций
 - ⦿ Как часто выполняются функции
 - ⦿ Построение графов вызовов
 - ⦿ Покрытие кода (выявление неиспользуемых фрагментов кода)
 - ⦿ Использование памяти
- 📡 Профилер (profiler) – программа для профилирования
 - ⦿ cProfile – встроенный профилер для Python

Пример профилирования кода

```
import cProfile

def slow_function():
    # Медленный код
    sum(range(10**9))

profiler = cProfile.Profile()
profiler.enable()

slow_function() # Анализируемый код

profiler.disable()
profiler.print_stats(sort='cumulative') # Сортировка по общему времени
```

Вывод результатов измерения времени выполнения функции

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	8.886	8.886	profile.py:4(slow_function)
1	8.886	8.886	8.886	8.886	{built-in method builtins.sum}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

Тестирование

- 📡 Тестирование программного обеспечения (Software Testing) — это процесс проверки соответствия программы заданным требованиям, поиска ошибок и оценки качества продукта
- 📡 Основные цели:
 - ⦿ Обнаружение ошибок
 - ⦿ Проверка функциональности
 - Работает ли система так, как задумано
 - ⦿ Оценка производительности
 - Скорость, стабильность, максимальная нагрузка
 - ⦿ Снижение рисков
 - Предотвращение сбоев в процессе эксплуатации
 - ⦿ Улучшение UX
 - Оценка удобства использования
- 📡 Тестирование — это неотъемлемая часть разработки, которая экономит время и деньги

Разновидности тестирования

По типу проверки

Тип	Что тестирует?
Функциональное	Соответствие требованиям (например, кнопка "Купить" должна работать).
Нагрузочное	Поведение под высокой нагрузкой (например, 1000 пользователей одновременно).
Юзабилити	Удобство интерфейса для пользователя.
Регрессионное	Проверка, что новые правки не сломали старый функционал.
Безопасность	Уязвимости (SQL-инъекции, XSS).

По уровню

Уровень	Что проверяет?
Модульное (Unit)	Отдельные функции/классы.
Интеграционное	Взаимодействие модулей между собой.
Системное	Работа всей системы в целом.

Юнит-тестирование

- 📡 Юнит-тестирование позволяет проверять корректность работы отдельных компонентов программы (функций, классов, методов) изолированно от остальных частей
- 📡 Юнит-тестирование программ на Python
 - 🕒 **unittest** – входит в стандартные библиотеки Python
 - Создавался по подобию JUnit (Java)
 - Требуется ООП программирования
 - Поддержка операций для подготовки и очистки после тестов
 - 🕒 **pytest** – более современный и простой каркас (требуется установки)
 - Минимум обязательного кода
 - Умное обнаружение тестов по именам файлов и функций
 - Параметризуемость и расширяемость тестов

Выражение assert

- Используется в отладке и тестировании для проверки условий, отражающих корректность выполнения программы
 - Если выражение после `assert` истинно, то ничего не происходит
 - Если выражение после `assert` ложно, то выбрасывается исключение `AssertionError`

```
assert x > 1
```

```
# Эквивалентно
```

```
if not x > 1 : raise AssertionError
```

- При запуске приложения в оптимизированном режиме (опция командной строки `-O`) проверки не выполняются и не замедляют выполнение программы

Каркас pytest

- Использует стандартное выражения **assert** для сравнения полученной и ожидаемой величин
- Тест – функция с именем, начинающимся на `'test_'` или заканчивающимся на `'_test'`, содержащая выражение **assert**
- Проверка исключений:
with pytest.raises(SomeError):
- Набор тестов – файл с именем `test_*.py` или `*_test.py`
- Запускается из командной строки (`pytest [options]`)
 - Запускает все наборы тестов в текущей директории и ее поддиректориях
- Выводит текстовые отчеты о результатах тестирования
- Имеет много возможностей конфигурирования тестов и создания образцов данных для тестирования

Пример: тестируемый код

Модуль calc.py

```
def sum(x, y):  
    return x + y  
  
def sub(x, y):  
    return x - y  
  
def prod(x, y):  
    return x * y  
  
def div(x, y):  
    return x / y
```

Функциональные требования:

- ⦿ Выполняет 4 арифметические операции

Пример: набор тестов

Файл test_calc.py

```
import pytest
import mod.calc as calc

def test_sum():
    assert calc.sum(1, 2) == 3

def test_sub():
    assert calc.sub(2, 1.1) == 0.9

def test_prod():
    assert calc.prod(2, 2) == 4

def test_div():
    assert calc.div(4, 2) == 2

def test_div_by_zero():
    with pytest.raises(ZeroDivisionError):
        calc.div(4, 0)
```

Пример: отчет о тестировании

Результат выполнения

```
D:\py-projects\programming>pytest
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-8.3.5, pluggy-1.5.0
rootdir: D:\py-projects\programming
collected 5 items

test_calc.py .F... [100%]

===== FAILURES =====
_____ test_sub _____

    def test_sub():
>         assert calc.sub(2, 1.1) == 0.9
E         assert 0.8999999999999999 == 0.9
E         + where 0.8999999999999999 = <function sub at 0x0000027F4F72BC40>(2, 1.1)
E         +   where <function sub at 0x0000027F4F72BC40> = calc.sub

test_calc.py:15: AssertionError
=====+===== short test summary info +=====
FAILED test_calc.py::test_sub - assert 0.8999999999999999 == 0.9
===== 1 failed, 4 passed in 0.11s =====
```

Test-Driven Development

Принцип:

- ⦿ Сначала пишется тест
- ⦿ Затем пишется код, который может пройти тест
- ⦿ Рефакторинг (улучшение кода без изменения функционала)

Результат:

- ⦿ Чистый, модульный код
- ⦿ Меньше ошибок выявляется на этапе эксплуатации

-  На основании функциональных требований пишутся наборы тестов, покрывающие все сценарии использования кода
- ⦿ При изменении требований изменяются и тесты

Итоги

В данной лекции были рассмотрены следующие темы:

-  Режим отладки
-  Типы отладчиков и профайлеров
-  Методологии тестирования ПО
-  Классификация тестов
-  Юнит-тестирование
-  Средства автоматизации тестирования